

# PKU–Zurich PhD Summer School on Machine Learning for Macroeconomics and Finance

ReLoBRaLo and gradient-conflict mitigation in multi-equation residual losses

Simon Scheidegger

HEC Lausanne · University of Zurich

July 6–10, 2026 · Beijing, China

[https://liboecon.com/2026\\_summer\\_school.html](https://liboecon.com/2026_summer_school.html)

# Roadmap of This Lecture

## I. Motivation

- ▶ Multi-component losses in PINNs and DEQNs
- ▶ The scale problem
- ▶ Multi-output architectures

## II. Balancing Methods

- ▶ Equal / manual weighting
- ▶ Inverse-loss weighting
- ▶ SoftAdapt
- ▶ **ReLoBRaLo** (+ finger exercise)
- ▶ GradNorm (brief)

## III. Experiments & Wrap-up

- ▶ Setup and results
- ▶ Sensitivity to temperature  $T$
- ▶ Connection to PINNs & DSGE
- ▶ Summary and references

Hands-on: 02\_03\_Loss\_Normalization.ipynb

# Learning objectives

**By the end of this lecture you should be able to:**

- ▶ Identify scale problems in multi-component losses (PINNs, DEQNs, multi-task) and diagnose gradient dominance
- ▶ Implement adaptive loss-balancing strategies: inverse-loss weighting, SoftAdapt, and ReLoBRaLo
- ▶ Derive and apply the ReLoBRaLo relative-ratio update rule with baseline mixing and exponential smoothing
- ▶ Compare GradNorm (gradient-norm balancing) with ReLoBRaLo and select appropriate methods for constrained optimization
- ▶ Tune temperature parameters and validate on multi-scale function approximation; apply to DEQN training with PINN-style residual losses

# Part I

Motivation

# Motivation: Multi-Component Losses

Many problems in economics require minimizing *several* loss terms simultaneously:

## Physics-informed NNs (PINNs):

- ▶ PDE residual  $\mathcal{L}_{\text{PDE}}$
- ▶ Boundary conditions  $\mathcal{L}_{\text{BC}}$
- ▶ Initial conditions  $\mathcal{L}_{\text{IC}}$

## Deep equilibrium nets (DEQNs):

- ▶ Euler equation residuals
- ▶ Budget constraints
- ▶ Market clearing

## Multi-task learning:

- ▶ Predict multiple outputs from shared features
- ▶ Each output has its own loss

## Core challenge

These loss components often live at **vastly different scales**.

# The Scale Problem

**Example:** Three loss components

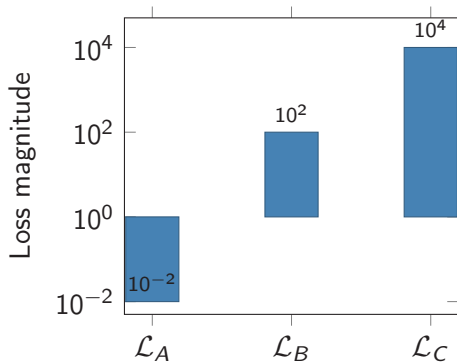
$\mathcal{L}_A \approx 10^{-2}$  (small-scale function)

$\mathcal{L}_B \approx 10^2$  (medium-scale)

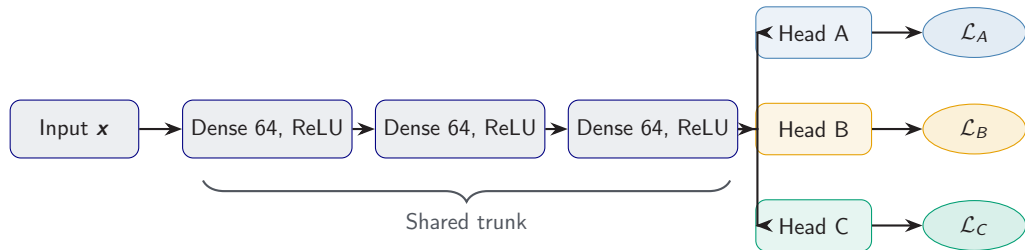
$\mathcal{L}_C \approx 10^4$  (large-scale)

With equal weights:  $\mathcal{L}_C$  contributes  $10^6 \times$  more gradient signal than  $\mathcal{L}_A$ .

$\Rightarrow$  The network *ignores* the small-scale component.



# Multi-Output Network Architecture



$$\mathcal{L}_{\text{total}} = w_A \cdot \mathcal{L}_A + w_B \cdot \mathcal{L}_B + w_C \cdot \mathcal{L}_C$$

How should we set the weights  $w_A, w_B, w_C$ ?

# Loss Balancing Strategies



# Equal Weighting

**Simplest approach:**  $w_A = w_B = w_C = 1$

$$\mathcal{L} = \mathcal{L}_A + \mathcal{L}_B + \mathcal{L}_C$$

**Problem:**

- ▶ If  $\mathcal{L}_C \gg \mathcal{L}_A$ , gradients are **dominated** by  $\mathcal{L}_C$
- ▶ Network learns to minimize  $\mathcal{L}_C$  first
- ▶  $\mathcal{L}_A$  barely decreases
- ▶ Small-scale structure is *ignored*

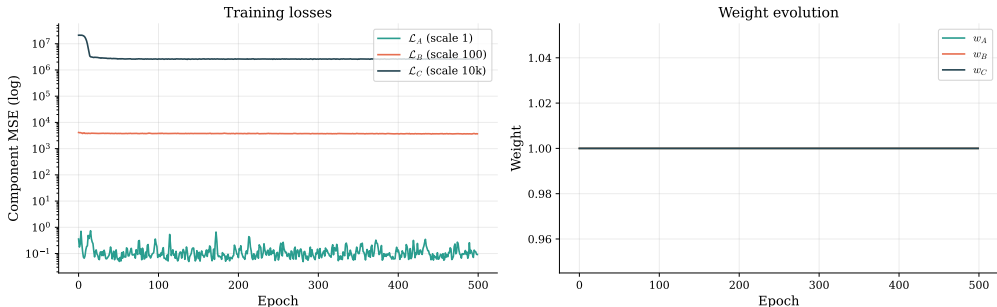
## Gradient dominance

$$\frac{\partial \mathcal{L}}{\partial \theta} \approx \frac{\partial \mathcal{L}_C}{\partial \theta} \quad \text{when } \mathcal{L}_C \gg \mathcal{L}_A, \mathcal{L}_B.$$

The network effectively solves a *single-task* problem.

# Equal Weighting — Training Dynamics

Equal Weighting



Generated by 02\_03\_Loss\_Normalization.ipynb (Section 3, equal-weight training). Left: per-component MSE on a log axis; right: the (constant) component weights.  $\mathcal{L}_C$  (scale  $10^4$ ) drives the gradient and converges quickly;  $\mathcal{L}_B$  (scale  $10^2$ ) follows;  $\mathcal{L}_A$  (scale  $10^0$ ) is **essentially not learned**, exactly as the analytical scale argument predicts.

# Manual Weighting

**Idea:** Choose weights to approximately equalize the loss magnitudes:

$$\mathcal{L} = w_A \cdot \mathcal{L}_A + w_B \cdot \mathcal{L}_B + w_C \cdot \mathcal{L}_C$$

e.g.,  $w_A = 10^4$ ,  $w_B = 10^0$ ,  $w_C = 10^{-2}$ .

## Pros:

- ▶ Can work well if scales are known a priori
- ▶ Simple to implement

## Cons:

- ▶ **Trial-and-error**: need to tune  $d$  extra hyperparameters
- ▶ **Fragile**: optimal weights change during training as losses evolve
- ▶ **Doesn't generalize**: weights are problem-specific

**Use adaptive methods that adjust weights during training.**

# Inverse-Loss Weighting

**Idea:** Weight each component inversely proportional to its current magnitude:

$$w_i^{(t)} = \frac{1}{\mathcal{L}_i^{(t)}}$$

This automatically up-weights small losses and down-weights large ones.

**With exponential averaging** (for stability):

$$\bar{\mathcal{L}}_i^{(t)} = \beta_{\text{ema}} \cdot \bar{\mathcal{L}}_i^{(t-1)} + (1 - \beta_{\text{ema}}) \cdot \mathcal{L}_i^{(t)}$$

$$w_i^{(t)} = \frac{1}{\bar{\mathcal{L}}_i^{(t)} + \epsilon}$$

**Pros:**

- ▶ Simple, no hyperparameters (besides  $\beta_{\text{ema}}$ )
- ▶ Adaptive to changing scales

**Cons:**

- ▶ **Unstable** when  $\mathcal{L}_i \rightarrow 0$
- ▶ Weights can **oscillate** wildly
- ▶ No notion of *relative progress*

# Inverse-Loss Weighting — Summary

## Algorithm

At each epoch  $t$ :

1. Compute current losses  $\mathcal{L}_1^{(t)}, \dots, \mathcal{L}_K^{(t)}$
2. Update exponential averages:  $\bar{\mathcal{L}}_i^{(t)} = \beta_{\text{ema}} \bar{\mathcal{L}}_i^{(t-1)} + (1 - \beta_{\text{ema}}) \mathcal{L}_i^{(t)}$
3. Set weights:  $w_i^{(t)} = \frac{1}{\bar{\mathcal{L}}_i^{(t)} + \epsilon}$
4. Normalize:  $\tilde{w}_i^{(t)} = K \cdot \frac{w_i^{(t)}}{\sum_{j=1}^K w_j^{(t)}}$

The normalization ensures  $\sum_i \tilde{w}_i = K$  (total weight is preserved).

Typical choices:  $\beta_{\text{ema}} = 0.99$ ,  $\epsilon = 10^{-8}$ .

# SoftAdapt

**Idea:** Weight based on *relative loss changes* — components that are improving slowly get higher weight.

## Rate of change

$$\Delta_i^{(t)} = \mathcal{L}_i^{(t)} - \mathcal{L}_i^{(t-1)}$$

- ▶  $\Delta_i > 0$ : loss *increased*  $\Rightarrow$  needs more weight
- ▶  $\Delta_i < 0$ : loss *decreased*  $\Rightarrow$  already learning well

## Softmax temperature $T$ :

- ▶ Small  $T$ : winner-take-all
- ▶ Large  $T$ : nearly equal weights

## Pros:

- ▶ Focuses on *struggling* components
- ▶ Simple to implement

## Cons:

- ▶ Sensitive to  $T$
- ▶ Noisy loss changes  $\Rightarrow$  noisy weights

# SoftAdapt — Formula

Weight update (Heydari et al., 2019)

$$\hat{w}_i^{(t)} = \frac{\exp(\Delta_i^{(t)} / T)}{\sum_{j=1}^K \exp(\Delta_j^{(t)} / T)} \cdot K$$

where  $\Delta_i^{(t)} = \mathcal{L}_i^{(t)} - \mathcal{L}_i^{(t-1)}$  and  $T > 0$  is a temperature parameter.

Interpretation:

- ▶ The softmax converts relative loss changes into a probability-like weighting
- ▶ Multiplication by  $K$  ensures  $\sum_i \hat{w}_i = K$  (preserves total scale)
- ▶ Components with **increasing loss** (poor progress) receive more weight
- ▶ Components with decreasing loss receive less weight

Scale caveat

Raw  $\Delta_i^{(t)}$  is dimensional: a component with loss  $\sim 10^3$  produces fluctuations that swamp one with loss  $\sim 10^{-3}$ . In practice rescale before the softmax, e.g.  $\tilde{\Delta}_i^{(t)} = \Delta_i^{(t)} / (\mathcal{L}_i^{(t)} + \varepsilon)$ , so the mechanism reacts to *relative* progress rather than absolute magnitudes.

# ReLoBRaLo — Relative Loss Balancing with Random Lookback

**Key innovations** (Bischof & Kraus, 2025), adapted here as a deterministic classroom version:

1. **Relative balancing:** Use *ratios*  $\mathcal{L}_i^{(t)} / \mathcal{L}_i^{(t-1)}$  instead of absolute values
2. **Baseline comparison:** Mix in the **initial losses**  $\mathcal{L}_i^{(0)}$  as a stable reference
3. **Exponential smoothing:** Blend new weights with history via  $\alpha$

## Why random lookback?

- ▶ Prevents weight drift
- ▶ “Resets” to global perspective
- ▶ Stabilizes training
- ▶ Stochastic regularization of the baseline-vs-step trade-off



# ReLoBRaLo — Algorithm

## Weight update at epoch $t$

- 1: Compute losses  $\mathcal{L}_1^{(t)}, \dots, \mathcal{L}_K^{(t)}$
- 2: **Step-wise ratios:**  $r_{i,s}^{(t)} = \mathcal{L}_i^{(t)} / (T \mathcal{L}_i^{(t-1)} + \varepsilon)$
- 3: **Baseline ratios:**  $r_{i,b}^{(t)} = \mathcal{L}_i^{(t)} / (T \mathcal{L}_i^{(0)} + \varepsilon)$
- 4:  $\hat{w}_{i,s}^{(t)} = K \text{ softmax}_i(r_{\cdot,s}^{(t)})$ ,  $\hat{w}_{i,b}^{(t)} = K \text{ softmax}_i(r_{\cdot,b}^{(t)})$
- 5: **Combine with smoothing:**

$$w_i^{(t)} = \alpha \left[ \rho w_i^{(t-1)} + (1 - \rho) \hat{w}_{i,b}^{(t)} \right] + (1 - \alpha) \hat{w}_{i,s}^{(t)}$$

- ▶  $\alpha$ : smoothing — how much to trust the historical weights
- ▶  $\rho$ : baseline mix — relative weight on the initial-loss comparison
- ▶  $T$ : temperature — sharpness of the softmax

# Finger Exercise: Loss Balancing

## Exercise

A DEQN has three loss components at iteration  $t$ :  $\mathcal{L}_1^{(t)} = 10.0$ ,  $\mathcal{L}_2^{(t)} = 0.01$ ,  $\mathcal{L}_3^{(t)} = 1.0$ .

1. If you use **equal weights**  $w_i = 1/3$ , which loss component dominates the gradient? Is this desirable?
2. Compute the **inverse-loss weights**:  $w_i \propto 1/\mathcal{L}_i^{(t)}$ , normalized to sum to 1.
3. Which component now receives the highest weight? Why does this help training?

# Solution: Loss Balancing

## Answer

1. With equal weights,  $\mathcal{L}_1 = 10.0$  dominates ( $100\times$  larger than  $\mathcal{L}_2$ ). The optimizer focuses almost entirely on reducing  $\mathcal{L}_1$  while ignoring  $\mathcal{L}_2$ . This is undesirable if all three conditions must be satisfied simultaneously.
2. Unnormalized:  $1/10.0 = 0.1$ ,  $1/0.01 = 100$ ,  $1/1.0 = 1$ . Sum = 101.1.  
Normalized:  $w_1 = 0.001$ ,  $w_2 = 0.989$ ,  $w_3 = 0.010$ .
3.  $\mathcal{L}_2$  receives the highest weight ( $\approx 99\%$ ). Inverse-loss weighting amplifies the components that are *hardest to satisfy* (smallest loss = most satisfied  $\Rightarrow$  smallest weight), directing optimization effort where it is most needed.

# ReLoBRaLo — Hyperparameters

| Symbol   | Name         | Default | Effect                                                                                           |
|----------|--------------|---------|--------------------------------------------------------------------------------------------------|
| $T$      | Temperature  | 1.0     | Softmax sharpness: small $T \rightarrow$ winner-take-all; large $T \rightarrow$ uniform weights. |
| $\alpha$ | Smoothing    | 0.999   | Trust in historical weights; larger $\alpha \rightarrow$ slower adaptation.                      |
| $\rho$   | Baseline mix | 0.999   | Weight on initial-loss comparison (convex blend).                                                |

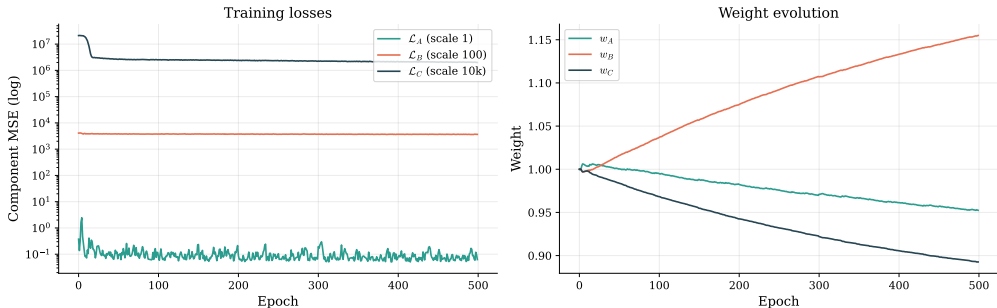
**Note.** The original algorithm uses a stochastic Bernoulli lookback of probability  $\rho$ ; the classroom version is a deterministic convex blend (simpler to debug).  $T$ ,  $\alpha$ ,  $\rho$  are *balancing hyperparameters* from the ReLoBRaLo literature — not Session 1's economic symbols ( $\alpha$  capital share,  $\beta$  discount factor); likewise  $\mathcal{L}_i$  is a loss *component*, Session 1's scalar loss  $\ell_\theta$  being the single-component case.

## Practical guidance

Defaults work well in most cases. Only  $T$  needs occasional tuning (see sensitivity analysis in Notebook 02\_03).

# ReLoBRaLo — Weight Evolution

ReLoBRaLo ( $T=1.0$ )



Generated by 02\_03\_Loss\_Normalization.ipynb (Section 5, ReLoBRaLo at  $T = 1$ ). Left: per-component MSE on a log axis under ReLoBRaLo (compare with the equal-weighting plot on slide 7). Right: the actual weight trajectories  $w_A, w_B, w_C$  produced by the ReLoBRaLo update rule.  $w_A$  rises (small-scale loss now amplified),  $w_C$  falls (large-scale loss attenuated), exactly as the algorithm targets.

# GradNorm (Brief)

**Idea** (Chen et al., 2018): Balance the *gradient norms* across tasks, not the losses.

## Objective

Adjust weights  $w_i$  so that:

$$\|\nabla_{\theta} w_i \mathcal{L}_i\|_2 \approx \bar{G} \cdot r_i^{(t)}$$

where  $\bar{G}$  is the average gradient norm and  $r_i^{(t)}$  captures relative training rates.

The weights  $w_i$  are **learnable parameters** updated via gradient descent on a “gradient balance” loss.

## Pros:

- ▶ Principled — directly addresses gradient competition
- ▶ Automatic, no manual tuning

## Cons:

- ▶ Requires per-task gradient computation (expensive)
- ▶ More complex to implement
- ▶ Hyperparameter  $\alpha$  (asymmetry)

# GradNorm — Formula

Gradient normalization loss (Chen et al., 2018)

$$\mathcal{L}_{\text{grad}} = \sum_{i=1}^K \left| \|\nabla_{\theta} w_i \mathcal{L}_i\|_2 - \bar{G} \cdot r_i^{(t)} \right|$$

where:

- ▶  $\bar{G} = \frac{1}{K} \sum_i \|\nabla_{\theta} w_i \mathcal{L}_i\|_2$  — average gradient norm
- ▶  $r_i^{(t)} = (\tilde{\mathcal{L}}_i^{(t)} / \tilde{\mathcal{L}}^{(t)})^{\alpha}$  — relative inverse training rate
- ▶  $\tilde{\mathcal{L}}_i^{(t)} = \mathcal{L}_i^{(t)} / \mathcal{L}_i^{(0)}$  — loss ratio relative to initial
- ▶  $\alpha$  controls asymmetry ( $\alpha = 1.5$  typical)

**Update:**  $w_i \leftarrow w_i - \eta_w \cdot \nabla_{w_i} \mathcal{L}_{\text{grad}}$ , then renormalize to  $\sum_i w_i = K$ .

# Experiment: Multi-Scale Function Approximation



# Experiment Setup

**Three target functions on  $[0, 1]^2$ :**

$$f_A(\mathbf{x}) = \text{genz\_gaussian}(\mathbf{x}) \\ \text{scale} \in [0, 1]$$

$$f_B(\mathbf{x}) = 100 \times \text{genz\_oscillatory}(\mathbf{x}) \\ \text{scale} \in [-100, 100]$$

$$f_C(\mathbf{x}) = 10,000 \times \text{genz\_continuous}(\mathbf{x}) \\ \text{scale} \in [0, 10,000]$$

**Architecture:**

- ▶ Shared trunk:  $3 \times 64$  ReLU
- ▶ 3 output heads (Dense(1) each)
- ▶ Adam,  $\eta = 10^{-3}$

**Data:**

- ▶  $n_{\text{train}} = 1,000$
- ▶  $n_{\text{test}} = 2,000$
- ▶  $d = 2$

$$\mathcal{L} = w_A \cdot \text{MSE}(f_A, \hat{f}_A) + w_B \cdot \text{MSE}(f_B, \hat{f}_B) + w_C \cdot \text{MSE}(f_C, \hat{f}_C)$$

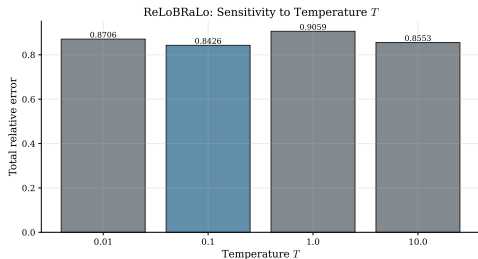
# Results: Equal vs. Inverse-Loss vs. ReLoBRaLo

|                                                           | Equal weights        | Inverse-loss         | ReLoBRaLo            |
|-----------------------------------------------------------|----------------------|----------------------|----------------------|
| $f_A$ mean error                                          | $3.3 \times 10^{-1}$ | $7.1 \times 10^{-3}$ | $1.6 \times 10^{-1}$ |
| $f_B$ mean error                                          | $5.4 \times 10^1$    | $3.4 \times 10^1$    | $3.1 \times 10^1$    |
| $f_C$ mean error                                          | $1.2 \times 10^3$    | $4.1 \times 10^3$    | $6.3 \times 10^2$    |
| Total rel. error $\sum_i  \overline{e_i} /\text{scale}_i$ | 0.99                 | 0.76                 | 0.53                 |

- **Equal:**  $f_C$  dominates the gradient;  $f_A$  is the **least accurate** run and the total error is highest.
- **Inverse-loss:** sharply improves  $f_A$  and roughly halves the total error, but  $f_C$  gets noticeably worse.
- **ReLoBRaLo:** best on  $f_B$  and  $f_C$  and the **lowest total relative error** ( $\sim 1.9\times$  below equal weighting);  $f_A$  is fit well, though less aggressively than inverse-loss.

Numbers from 02\_03\_Loss\_Normalization.ipynb, RUN\_MODE = "teaching" (500 epochs), seed 42. Absolute values are seed-/mode-sensitive; the qualitative point is that adaptive weighting lowers the scale-weighted total.

# Sensitivity to Temperature $T$



Generated by 02\_03\_Loss\_Normalization.ipynb (Section 7, temperature sweep). Bars show the total relative error  $\sum_i \text{MAE}_i / \text{scale}_i$  at the end of training, swept over  $T \in \{0.01, 0.1, 1.0, 10.0\}$ .

$T \approx 1$  is a robust default; sensitivity analysis is cheap.

$T$  too small ( $\ll 1$ ):

- ▶ Winner-take-all
- ▶ Only one component trained at a time
- ▶ Slow convergence

$T$  too large ( $\gg 1$ ):

- ▶ Near-equal weights
- ▶ Reverts to the scale problem
- ▶ No effective balancing

# Connection to PINNs and DSGE

## Physics-informed NNs:

- ▶ PDE residual:  $\mathcal{L}_{\text{PDE}} \sim \mathcal{O}(1)$
- ▶ Boundary conditions:  $\mathcal{L}_{\text{BC}} \sim \mathcal{O}(10^{-2})$
- ▶ Initial conditions:  $\mathcal{L}_{\text{IC}} \sim \mathcal{O}(10^{-4})$
- ▶ ReLoBRaLo ensures BCs/ICs are **not ignored**

## DSGE / Deep equilibrium nets:

- ▶ Euler equations at different scales
- ▶ Budget constraint residuals
- ▶ Market clearing conditions
- ▶ Each may have different units and magnitudes

## Bischof & Kraus (2025)

ReLoBRaLo was originally developed for PDE-constrained optimization and has been shown to improve convergence by up to an order of magnitude on multi-physics problems.

# Summary

## Key takeaways:

1. Multi-component losses at different scales cause **gradient dominance**.
2. Equal weighting fails when scales differ by  $> 10\times$ .
3. Inverse-loss weighting helps but can be unstable.
4. **ReLoBRaLo** provides robust, adaptive balancing via relative ratios + baseline mix.
5. Critical for PINNs, DEQNs, and multi-task learning in economics.

## References:

- ▶ Bischof & Kraus (2025). *Multi-Objective Loss Balancing for Physics-Informed Deep Learning*, CMAME 439, 117914.
- ▶ Chen et al. (2018). *GradNorm*, ICML.
- ▶ Heydari et al. (2019). *SoftAdapt*, arXiv:1912.12355.

## Hands-on

⇒ `02_03_Loss_Normalization.ipynb`

Applied exercise: `code/04_irbc/04_03_IRBC_Exercise.ipynb` uses inverse-loss weighting on the IRBC model of Session 4 — a direct preview of ReLoBRaLo in a DEQN.